

Table des matières

1. Pour commencer
2. Premiers pas avec les fichiers bas-niveau
3. Déplacements dans un fichier
4. Copie de fichier

1. Pour commencer

Le but de ce TP est de manipuler les fonctions bas-niveau associées aux fichiers. Dans tous vos codes, vous devez tester systématiquement **tous les retours** des appels système.

Même si vous ne terminez tous les exercices lors de cette séance, il est fortement conseillé de le terminer **avant** le TP prochain. Certaines parties seront réutilisées dans les prochains TPs... sans oublier le QCM !

2. Premiers pas avec les fichiers bas-niveau

Dans cet exercice, nous allons exploiter les fonctions de base : `open`, `close`, `read` et `write`.

1. Faites un programme qui demande à l'utilisateur de saisir un entier et qui le sauvegarde dans le fichier `toto.bin`.
2. À l'aide de la commande `od`, vérifiez le contenu du fichier. À quoi correspondent les suites de '00' ?

Pour visualiser le contenu d'un fichier binaire avec `od`, tapez la commande suivante : `od -t x1 toto.bin`

3. Que se passe-t-il si vous ouvrez le fichier avec un éditeur de code "classique" ?
4. Modifiez votre programme et écrivez maintenant une chaîne de caractères (lue au clavier) après l'entier. Observez le contenu du fichier avec `od` et votre éditeur de code.

Dans la suite, si vous ne souhaitez pas utiliser `od`, installez l'extension d'édition hexadécimale dans votre éditeur.

3. Déplacements dans un fichier

La fonction `lseek` permet de se déplacer dans un fichier. Elle retourne la nouvelle position dans le fichier (ou -1 en cas d'erreur).

1. Comment utiliser `lseek` pour connaître la position courante dans un fichier sans se déplacer ?
2. Utilisez `lseek` pour déterminer la taille d'un fichier dont le nom est passé en argument.
3. Écrivez un programme qui sauvegarde les valeurs de 1 à 10 (`int`) dans un fichier. Affichez ensuite la position actuelle.
4. Modifiez votre programme : une fois les entiers sauvegardés, déplacez-vous à la position 1024. Que se passe-t-il ? Quelle est la taille du fichier ?
5. Et si vous écrivez maintenant un caractère à la position 1024 ?
6. Utilisez la commande `stat` sur le fichier généré.

4. Copie de fichier

Le but de cet exercice est de réaliser une copie d'un fichier quelconque (fichier texte ou binaire) dans un autre fichier.

1. Écrivez un programme permettant de tester l'existence d'un fichier dont le nom est passé en argument, en **utilisant uniquement** l'appel système `open` (en utilisant les bonnes options et la gestion d'erreurs avec `errno`).
2. Modifiez le programme. Il prend maintenant **en arguments** le nom d'un fichier source et le nom d'un fichier de destination. Si le fichier source n'existe pas ou si le fichier de destination existe déjà, une erreur est affichée. Le fichier source ne doit pas être créé (dans cette question).
3. Comment copier un fichier dans un autre sans placer l'intégralité de son contenu en mémoire ?
4. Modifiez le programme pour qu'il procède à la copie (sans placer l'intégralité du contenu du fichier source en mémoire). On utilisera un tampon de taille N (où N est une constante de valeur 255).

Pour vérifier que la copie s'est bien déroulée, commencez par comparer la taille des fichiers. Vous pouvez utiliser également la commande `diff`.

La commande `time` permet de calculer le temps d'exécution d'un programme. Pour cela, il suffit de passer le nom du programme et ses arguments comme arguments de la commande.

5. Calculez le temps d'exécution nécessaire pour réaliser la copie du fichier.
6. Fixez la valeur de N à 1 puis comparez.